Mini-Project: Bank Account System

Objective

Design a multi-level inheritance hierarchy representing different types of bank accounts, and use calculations specific to each account type.

Description

Students will create a simple bank account system with three classes in a multi-level hierarchy:

1. **Account** (Base Class)
    - Attributes: accountNumber, balance
    - Methods: deposit(double amount), withdraw(double amount), calculateMonthlyFee(), displayAccountInfo()
2. **SavingsAccount** (Derived from Account)
    - Attributes: interestRate
    - Override:
        - calculateMonthlyFee(): Calculate no monthly fee for a savings account.
        - calculateMonthlyInterest(): Calculate interest based on the balance and interest rate.
        - displayAccountInfo(): Override to display interest rate along with account info.
3. **CheckingAccount** (Derived from SavingsAccount)
    - Attributes: overdraftLimit
    - Override:
        - withdraw(double amount): Add overdraft protection by allowing withdrawals up to the overdraft limit.
        - calculateMonthlyFee(): Calculate a monthly fee specific to checking accounts.
        - displayAccountInfo(): Override to display overdraft limit along with account info.

Instructions

1. **Create the Classes**:
    - Implement the Account, SavingsAccount, and CheckingAccount classes. Each class should initialize its own fields, with the base class fields being passed using super().
2. **Override Methods**:
    - **deposit(double amount)**:

        - Account: Increase the balance by the deposit amount.
    - **withdraw(double amount)**:
        - Account: Decrease the balance by the amount, if sufficient funds are available.
        - CheckingAccount: Allow withdrawals even if the balance is below zero, up to the overdraft limit.
    - **calculateMonthlyFee()**:
        - Account: General method that could be overridden by subclasses if different fee calculations are needed.
        - CheckingAccount: Override to calculate a monthly fee specific to checking accounts.
    - **displayAccountInfo()**:
        - Each class should override this method to display all relevant details of the account type.
3. **Write a Main Program**:
    - In a main class (e.g., BankMain), create instances of Account, SavingsAccount, and CheckingAccount.
    - Test each method, including deposits, withdrawals, and monthly fee/interest calculations, to verify correct functionality.

Questions for Students

After completing the code, students should answer the following questions to deepen their understanding:

1. **Multi-Level Inheritance**:
    - Explain how multi-level inheritance is applied in this project.
    - Describe the relationships between Account, SavingsAccount, and CheckingAccount.
2. **Method Overriding**:
    - Why did you override methods like withdraw() and calculateMonthlyFee() in CheckingAccount?
    - How does overriding displayAccountInfo() in each class help show the specific details for each account type?
3. **Using super()**:
    - Why is super() used in the constructors? Explain its importance when initializing inherited fields.
4. **Calculations**:

- What unique calculations are performed in each class?
- How does the use of different calculations in each class demonstrate the need for class-specific methods?

5. **Adding More Account Types**:
    - Imagine adding a BusinessAccount class. What attributes and overridden methods might you add to this class? How would this design support easy extension?

Submission:

1. Source Code:

- **All Java files** (Account.java, SavingsAccount.java, CheckingAccount.java, and BankMain.java or similar), containing the implementation of each class with constructors, overridden methods, and any additional logic specified in the project description.
- The code should be well-organized, commented, and formatted for readability.

2. Project Report:

- **Sample Output**: Screenshots or output examples showing sample runs of the program, including:
    - Creation of instances of Account, SavingsAccount, and CheckingAccount.
    - Example deposits, withdrawals, fee calculations, and results of displayAccountInfo() for each account type.
- **Reflection Questions**: Answers to reflection questions provided:
    - Explanation of the multi-level inheritance structure and why each class is structured as it is.
    - Discussion on the benefits of method overriding in this project.
    - Suggestions for potential extensions, like a BusinessAccount class.

3. Optional - UML Diagram:

- **Class diagram** (optional, but encouraged) showing the inheritance hierarchy.
    - This could be created using software like Lucidchart, Draw.io, or similar.
    - The diagram should include classes, attributes, and methods to illustrate the relationships among Account, SavingsAccount, and CheckingAccount.